TRI10NWARE

Acceptance criteria and developer handoff

Approved rules consolidated October 12, 2026
Targets: macOS, Windows, and desktop browser

Purpose: establish observable behavior, repeatable checks, and delivery evidence for a precision platformer inspired by *Celeste*. This is a specification, not a claim that any build has passed testing.

Authority and scope

Use this document with the October 12, 2026 TRI10NWARE design PDF. The acceptance decisions here supersede earlier deferred or provisional wording on the same topic. Original mockups remain visual/traversal references. Later owner-supplied room mockups define their geometry and content; they do not silently replace these system rules.

Gate	Required delivery
Level 1	A playable reproduction of the first mockup crossing; its object begins powered. Full base movement kit, state switching, charged jump, volts/countdown, retries, presentation, and cross-platform delivery.
Later core rooms	Owner-supplied mockups introducing state-dependent platforms, contact-activated networks, collectibles, and interconnected-room progression. Apply core-system criteria when each system enters scope.
Complete jam game	All agreed rooms and core systems integrated, with a clear completion condition, usable backtracking, consistent resets, and the full delivery/evidence package on all targets.

Optional features are excluded from required scope.
Water, viruses, propulsion objects, timed/interfering networks, and whole-wall phasing require a later explicit feature decision. No additional story development or geometry-manipulation system is required.

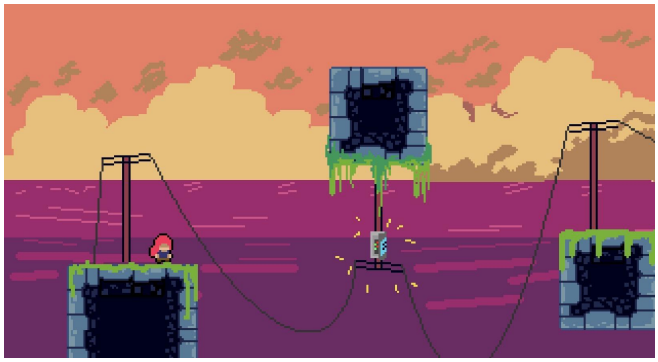
What counts as acceptance

- Every criterion applicable to the milestone passes and has reproducible evidence. Missing or inconclusive evidence is not a pass.
- The developer reports defects, decisions, tuning, support coverage, and deviations in the initial submission; no clarification interview is assumed.
- The owner verifies the build and explicitly approves movement feel and visual quality. Developer self-assessment and a video alone do not establish acceptance.

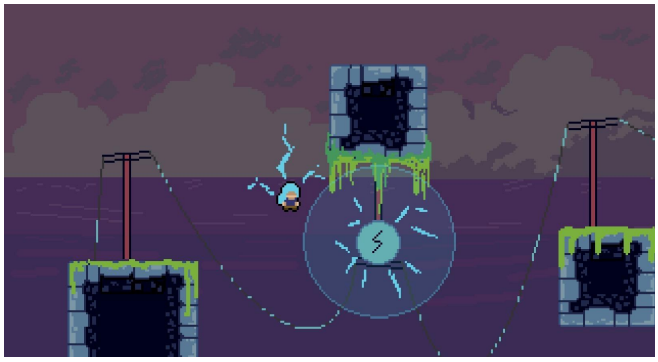
The owner supplies later room layouts/counts before their milestones. This specification does not invent those rooms. Remaining bounded developer choices and final content gates are listed on page 9.

Level 1: room and visual reference

The first room proves: analyze the gap, enter flow, jump toward the powered object, trigger an aerial charged jump, and land on the destination. Network-root activation is not required in Level 1.



Original default-state mockup



Original flow-state mockup

ID	Required behavior and reproducible check
L1-01	**Faithful room relationships.** Show the starting platform, opposing destination, overhead geometry, hanging electrical object, and visible electrical connections as depicted. Compare the playable room with both references. Preserve pixel-art direction and relative traversal relationships; the image does not specify exact pixel coordinates.
L1-02	**Powered from the start.** On a fresh start and every retry, the object is powered. The player can attempt the charged crossing immediately without activating a root or finding a recharge source.
L1-03	**Intended traversal works.** Demonstrate the complete crossing through ordinary jump, flow entry, valid charged jump, and safe destination landing. It must not require clipping, render-frame-dependent input, invisible footing, or another exploit.
L1-04	**Completion and failure.** Safe destination landing visibly completes Level 1; side contact or flying past does not. Missing the crossing and falling below the documented failure boundary triggers death and a clean retry, not an endless fall.
L1-05	**Level 1 and test coverage are separate.** The build includes the full base kit. If wall jumps, dash directions, or edge cases cannot be verified naturally in this room, provide a developer verification scene. It does not count as an additional finished game room.

Geometry and tuning discretion

The developer selects documented physics values and may make necessary scale/spacing adjustments to achieve the illustrated crossing. Disclose adjustments with the reference comparison. Ordinary-jump or dash bypasses must be reported; a blanket ban on all advanced alternate routes has not been approved. The charged route must remain a practical, demonstrable solution.

Movement, input, and dash

ID	Required behavior and reproducible check
M-01	**Ordinary movement.** A single ordinary jump and controllable horizontal movement work on ordinary ground. An ordinary jump cannot be repeatedly triggered in open air without another permitted opportunity. Show takeoff, apex, landing, turning, and stopping.
M-02	**Wall jump.** Ordinary solid walls support the documented wall-jump behavior in both states. Test both wall sides. Contact alone does not restore dash. Repeat inputs must not cause clipping or unexplained loss of control.
M-03	**Directional dash.** Dash works on ground and in air in default and flow. Test horizontal, vertical, and diagonal directions; document the direction scheme and no-direction fallback. Show collision with floor, ceiling, and wall. No unintended tunneling or diagonal speed multiplier.
M-04	**One available dash.** After use, another ordinary dash is unavailable until restored. Landing on supporting ground or successfully using a charged object restores one dash; merely overlapping a charged region, touching a wall, or changing state does not.
M-05	**Coyote time: 100 ms.** Step off a ledge and press jump within the configured window: an ordinary jump succeeds. Outside it, the grounded jump opportunity has expired. A consumed opportunity cannot be used twice.
M-06	**Jump buffer: 100 ms.** Press jump shortly before landing: it executes at the first eligible landing within the window. An expired press does not execute. One press produces one action; a consumed grounded press cannot also produce a charged jump.
M-07	**Inputs remain reliable.** Fresh presses are distinct from holding a button. Test press/release, rapid direction changes, and simultaneous jump/dash/switch input. Record precedence and demonstrate one consistent result rather than double execution.
M-08	**No movement volt cost.** With a known volt count, repeat jumps, wall jumps, and dashes without entering flow again. Volts remain unchanged.

Timing and feel acceptance

Measure input windows in simulation time. Boundary tests may differ by at most one physics tick due to sampling; report tick rate and measurements. The developer records ordinary jump-height control, acceleration, gravity, speed, dash duration/cooldown, and steering rules. Charged-jump rules on page 4 are fixed independently of ordinary-jump tuning.

Owner playtest: takeoff, steering, turning, landing, and dash timing must feel responsive and predictable. Animation/effects must not conceal an input delay or add undocumented control lockout. Demonstrate a clean crossing and a recoverable failed attempt; identify whether each failure is timing, positioning, or resource exhaustion.

Charged-jump eligibility and launch

ID	Required behavior and reproducible check
C-01	**Non-solid interaction region.** The powered object offers an aerial interaction region. While inside it in flow, an eligible jump press can activate while rising, at the apex, or falling. Landing on the object is not required. Region size/shape and overlap boundary are documented and visible in debug mode.
C-02	**State and power gates.** Default cannot execute a charged jump. An unpowered object cannot grant one in either state. Passing through a powered region without a valid jump press produces no automatic launch or passive dash refresh.
C-03	**One jump per encounter.** A successful charged jump consumes that object's encounter eligibility. A second fresh press while still inside produces no second charged jump. Holding the button also produces no repeated launch.
C-04	**Physical re-entry restores eligibility.** Leave the region and re-enter: another charged jump is available. Repeat on the same object. Toggling default/flow while remaining inside does not reset consumed eligibility. Each object tracks its own encounter independently.
C-05	**Consistent vertical launch.** Charged jump replaces incoming vertical velocity with its configured upward launch speed; it does not add to incoming speed. Test rising, near-zero, and fast-falling approaches. Launch speed is the same within one-tick measurement tolerance.
C-06	**Horizontal motion and steering.** Preserve incoming horizontal velocity at launch and retain normal air steering afterward. Releasing jump does not shorten this charged jump. Obstacles still stop upward movement normally; a ceiling hit does not restore encounter eligibility.
C-07	**Buffered charged input.** A fresh press up to 100 ms before entry executes when power, flow, airborne overlap, and encounter eligibility first become valid. If eligibility does not occur before expiry, no charged action executes. Consume the press once.
C-08	**Refresh and precedence.** Successful charged use restores dash availability. Show the dash value before/after a charged jump. Document and test dash-from-object behavior, simultaneous jump/dash priority, and refresh ordering; never award multiple actions from one consumed press.

Measure launch speed, not identical world-space peaks.
Different activation heights and later steering produce different trajectories. Verify equal vertical speed immediately after launch under equivalent physics conditions, with ceiling contact excluded from that measurement.

For C-04, repeatedly returning is allowed. A true geometric exit is required; changing state alone is not an exit. Test overlap near the boundary and show that stationary contact does not falsely re-arm due to collision/filter changes.

States, volts, countdown, and retries

ID	Required behavior and reproducible check
S-01	**Two states.** Default and flow are distinguishable in both visuals and debug state. Switching works grounded, airborne, and during dash, preserving position/velocity and the active dash trajectory; it does not grant an extra jump or dash.
S-02	**Entry cost.** Default to flow costs exactly one volt, once per transition. Flow to default costs none. Maximum is 3; values never become negative or exceed 3. Holding switch does not generate repeated transitions/costs.
S-03	**Last volt remains usable.** Enter flow with one volt: reach zero, start a visible three-second countdown, and retain flow interactions. Exiting to default does not stop or reset the countdown. Attempted default-to-flow entry at zero is blocked without changing state or timer.
S-04	**Recharge and expiry.** Successful network activation restores 3 volts and cancels the countdown. Without recovery, expiry kills the player. No duplicate death, residual timer after recovery, or second timer from repeat input. Document a deterministic tie rule when valid recharge and expiry occur on the same tick.
R-01	**Complete retry reset.** Death restores the room-start/checkpoint position, default state, 3 volts, one available dash, zero velocity, fresh encounters, and no active countdown. Consumed input/old buffers cannot trigger an unintended action upon respawn.
R-02	**Fast control return.** From the death event, the player is alive and controllable within one second. No manual reload or menu navigation is needed to retry the failed crossing.
R-03	**Level 1 retry.** Its object remains powered on every retry. Repeat at least 20 deaths/restarts including failed jumps and countdown deaths. No changed launch behavior, accumulating timers/effects, duplicate objects, or stuck input.
R-04	**Pause/focus behavior.** Pause or focus loss cannot silently expire a recovery countdown or leave keys stuck. The developer documents how simulation pauses/resumes, clears held-input state safely, and verifies native and browser behavior.

Starting state and temporary values

Level 1 starts with the same playable state as R-01 and a powered object. The 100 ms input windows and three-second countdown are approved initial targets and must be exposed as tuning values. Deliver the configured values and measurements. Any proposed change is a disclosed deviation, not a silent reinterpretation.

Level 1 has no required recharge root. Running out of volts must still lead to a clean retry. In later rooms, the developer must test final-volt recovery with a reachable root under the room's intended entry conditions.

Presentation and state-dependent platforms

Presentation: required in Level 1

ID	Required behavior and reproducible check
V-01	**Pixel-art fidelity.** Compare screenshots of both states with the original mockups. Keep crisp, stable pixel-art presentation across supported window sizes; no stretched geometry, blurred key objects, clipped HUD, or camera framing that conceals the intended crossing.
V-02	**Flow feedback.** Sprite/hair and a switch effect indicate state. Flow slightly dims the background and emphasizes electrical objects through glow/current effects. The player, surfaces, hazards, and destination remain readable.
V-03	**Action readability.** Show flow state, remaining volts, available dash, powered object, and countdown. The visible interaction cue corresponds to the usable charged region closely enough to anticipate activation; debug mode exposes its exact boundary.
V-04	**Feedback without obscuring play.** Charged jump, dash, death/retry, and room completion have clear visual/audio feedback. State-switch VFX coexist with movement animation. Critical status remains understandable with audio muted; effects do not block control or cover the landing.

Platforms: required when their core room enters scope

ID	Required behavior and reproducible check
P-01	**State matrix.** Ordinary ground/walls are solid in both states. Default-only platforms support default and are passable in flow. Flow-only platforms support flow and are passable in default. Demonstrate each cell of this matrix.
P-02	**One-way support.** State-dependent platforms initially support from above in the matching state. Ascending from below or passing through from the side does not become whole-wall collision. Ordinary walls never phase with switching.
P-03	**Immediate support change.** Switch to flow while standing on a default-only platform: support is removed and the player falls. Jump first, then switch: the jump continues. Test the opposite state/platform family and midair landings.
P-04	**Safe overlap handling.** Changing into a matching state during overlap cannot embed, teleport, crush, or catapult the player. Document the one-way landing/overlap policy and demonstrate rising, falling, edge, and dash-transition cases.

Owner quality review: a short playthrough must communicate the current state and why an interaction succeeded or failed. Color can support this, but animation/shape/effects or interface cues should also distinguish the important states. Rendering nodes, animation filenames, and a specific glow implementation are not acceptance requirements.

Networks and integrated progression

These are required core-game criteria, introduced through later owner-supplied room mockups. They are not additional prerequisites for Level 1. Demonstrate their rules in verification scenes before claiming system completion.

ID	Required behavior and reproducible check
N-01	**Contact activation.** Default contact does not activate an unpowered root. Flow contact does, whether approached by walking, jumping, falling, or dashing. Switching into flow while overlapping the root also follows the same contact rule. No compulsory dash impact or extra interaction button.
N-02	**Activation effects.** An eligible activation powers connected objects, restores volts to 3, and cancels a zero-volt countdown. Unrelated networks remain unchanged. Visual connections and power states communicate what became usable.
N-03	**Repeat contact and reset policy.** Use the design's working default: recharge occurs once per activation/reset period. Repeat contact with an already active root does not farm volts. Document exact room/checkpoint/death reset scope and demonstrate it; any departure is a declared design deviation.
N-04	**Death resets.** Death resets local network activation and retains defined permanent progress. Re-entry and checkpoint changes follow the documented policy consistently. Keep Level 1's deliberately powered starting object separate from resettable network activations.
G-01	**Backtracking without traps.** Test each connected room passage in both directions under supported world states, including after death/reset and with depleted resources. A player must have a valid route, recovery, or clean retry; no permanent progression trap.
G-02	**Permanent progress.** Identify which collectible/unlock flags persist through death and test each. Document pickup commitment rules and checkpoint effects. Do not confuse temporary electricity with permanent unlocks. Persistence after closing the app is a separate documented choice.
G-03	**Room feasibility and purpose.** For each required room, provide its intended entry state, volt budget, recharge points, and completion route. Demonstrate required jumps and transitions. Core challenges combine movement and switching; tutorials may use simple alternation.
G-04	**Exploration and completion.** Agreed rooms include the documented interconnected exploration and collectible challenges. Show a fresh-start route to the defined ending/completion signal, optional challenge access, backtracking, and completion after deaths. Match the owner's final room/content list.

No minimum room count, collectible count, playtime, or new ending/story content was specified in the discussion. These are supplied by the owner's later mockups and agreed content list. Do not invent them or claim the complete game is accepted while that scope remains undefined.

Platform delivery and technical reliability

ID	Required behavior and reproducible check
D-01	**Three delivery targets.** Supply runnable macOS and Windows builds plus a browser-playable web export. All represent the same source revision, milestone, rules, and tuning; a successful editor run does not replace exported-build checks.
D-02	**Support matrix.** State tested OS versions, CPU architectures, hardware, engine version, and desktop browser names/versions. Test native builds on their actual target OS and browser play on both macOS and Windows in the claimed matrix. Identify unsupported/unverified coverage explicitly.
D-03	**Independent launch.** A recipient can launch each build without the developer's machine, engine installation, credentials, or hidden paths. Supply complete assets and first-launch instructions. Browser export includes serving/hosting instructions and required files; validate through HTTP(S), not a local file URL.
D-04	**Equivalent behavior.** Run the intended crossing and state/resource/retry checks on every target. Geometry, launch speed, input eligibility, countdown, and completion behavior agree within physics-tick tolerance. Audio starts after the browser's user-gesture requirement where needed.
D-05	**Rendering-rate independence.** Where supported, compare 30, 60, and 120 rendering FPS using equivalent initial conditions and timed inputs. Record launch speed, jump displacement, dash distance, input windows, and countdown duration. Rendering FPS must not change rules or measured results beyond sampling tolerance.
D-06	**Performance evidence.** Report hardware, resolution, observed frame rate/frame times, and notable stalls during traversal and retries. The supported configuration must remain playable without sustained stalls that interrupt input or obscure timing. No universal minimum-hardware claim is implied.
D-07	**Window, focus, and browser checks.** Verify resize, fullscreen transitions where supported, keyboard focus, pause/resume, audio, and controls on each target. Browser navigation/scroll shortcuts must not interrupt normal supported gameplay inputs.
D-08	**Reproducible editable project.** Supply full Godot project/source, scenes, assets, dependencies, export instructions/presets, controls, and a source revision/build identifier. Opening and exporting it using the documented setup reproduces the submitted behavior. Include asset attribution/license information.

Cross-target bugs, missing assets, crashes, and reproducibility failures block the affected target's acceptance. Do not label an untested target passed. If a frame-rate condition cannot be reached, record the limitation and the conditions actually tested; it is not evidence of a pass at that rate.

Evidence package and reviewer decision

Submit once, with enough detail to reproduce

- **Builds and project:** all targets, one source revision, launch/export instructions, controls, tested support matrix, and verification-scene/debug access instructions.
- **Results ledger:** one row per criterion ID, milestone, target, starting conditions, exact input/action sequence, expected outcome, actual outcome, evidence reference, and Pass/Fail/Not tested/Out of milestone status.
- **Evidence:** short videos/screenshots for visual/manual cases; event traces or measurements for timings, velocities, costs, refreshes, and resets. Identify build and test setup. Include failed/negative cases as well as successful traversal.
- **Decision and tuning log:** selected values and every developer-owned choice below, with rationale and tests. No undisclosed change to a fixed rule.
- **Known issues:** severity, reproduction steps, affected targets/criteria, workaround if any, and whether the submitted build contains the issue. A reported blocker is still a blocker.

ID	Milestone / target	Test and result record
C-05	Level 1 / each target	Rising, apex, and falling launch: initial conditions, configured launch speed, measured post-action speed, tick rate, evidence, status.
R-03	Level 1 / each target	20 deaths/restarts: causes, reset state observations, timing, any accumulated/stuck behavior, evidence, status.

These are result-record templates, not completed tests. Apply the same structure to every applicable ID. Automated checks suit deterministic rules; manual reproduction and owner playtesting remain necessary for presentation and feel.

Bounded developer choices: document in the first submission

Document ordinary-jump height control; movement/gravity and wall-jump values; dash directions, duration/cooldown and fallback; region geometry; input precedence and charged-dash refresh order; recharge/expiry tie order; one-way overlaps; pause/focus; network reset scope; collectible commitment/session persistence; and target support. Preserve fixed rules and demonstrate each choice.

Acceptance decision

Blockers: crash, progression trap, invalid input ignored or duplicated, multiple charged jumps during one encounter, wrong volt/refresh/reset behavior, incompatible delivery, missing project assets, or absent evidence for a required behavior. Any failed applicable functional criterion remains unaccepted.

Owner review: approve responsive movement, useful charged traversal, readable state/object cues, mockup fidelity, and consistent retries. A functional pass does not automatically approve an unsatisfying movement result. Disclosed deviations and unresolved quality issues require an explicit owner decision; silence is not acceptance.

Final gate: an owner-approved room/content list and completion condition exist; every required room and criterion passes; fresh-start completion and reset/backtracking checks pass on all targets; no blockers remain. Optional features require separate commissioning.